

Master Seminar in

Collaborative Robotics and Assistive Technology for Advanced Human-Robot Interaction

Week 5

Introduction to **ROS** Robot Operating System

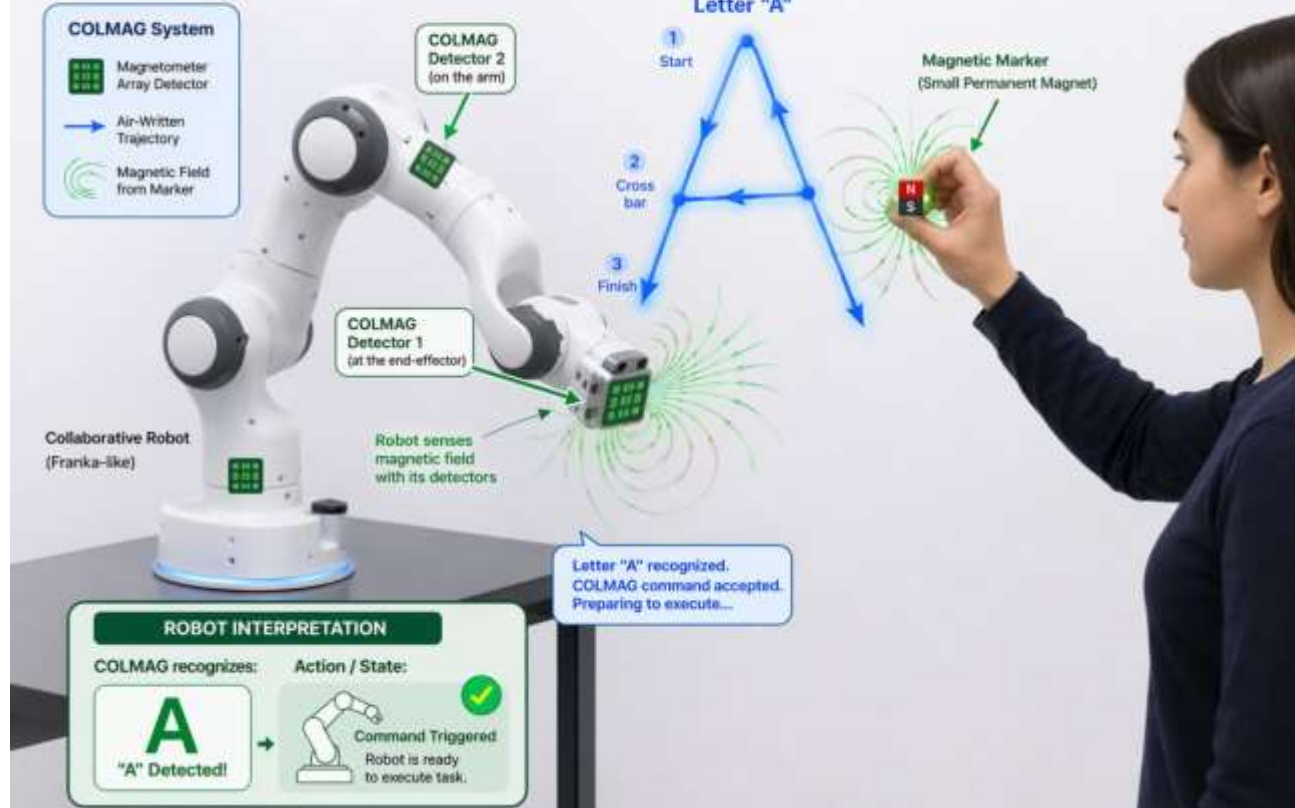
F. Masiero, E. Aimi, L. Borriello, Prof. L. Masia

Aries Lab – Technical University of Munich

COLMAG: Air-Writing with Magnetic Fields

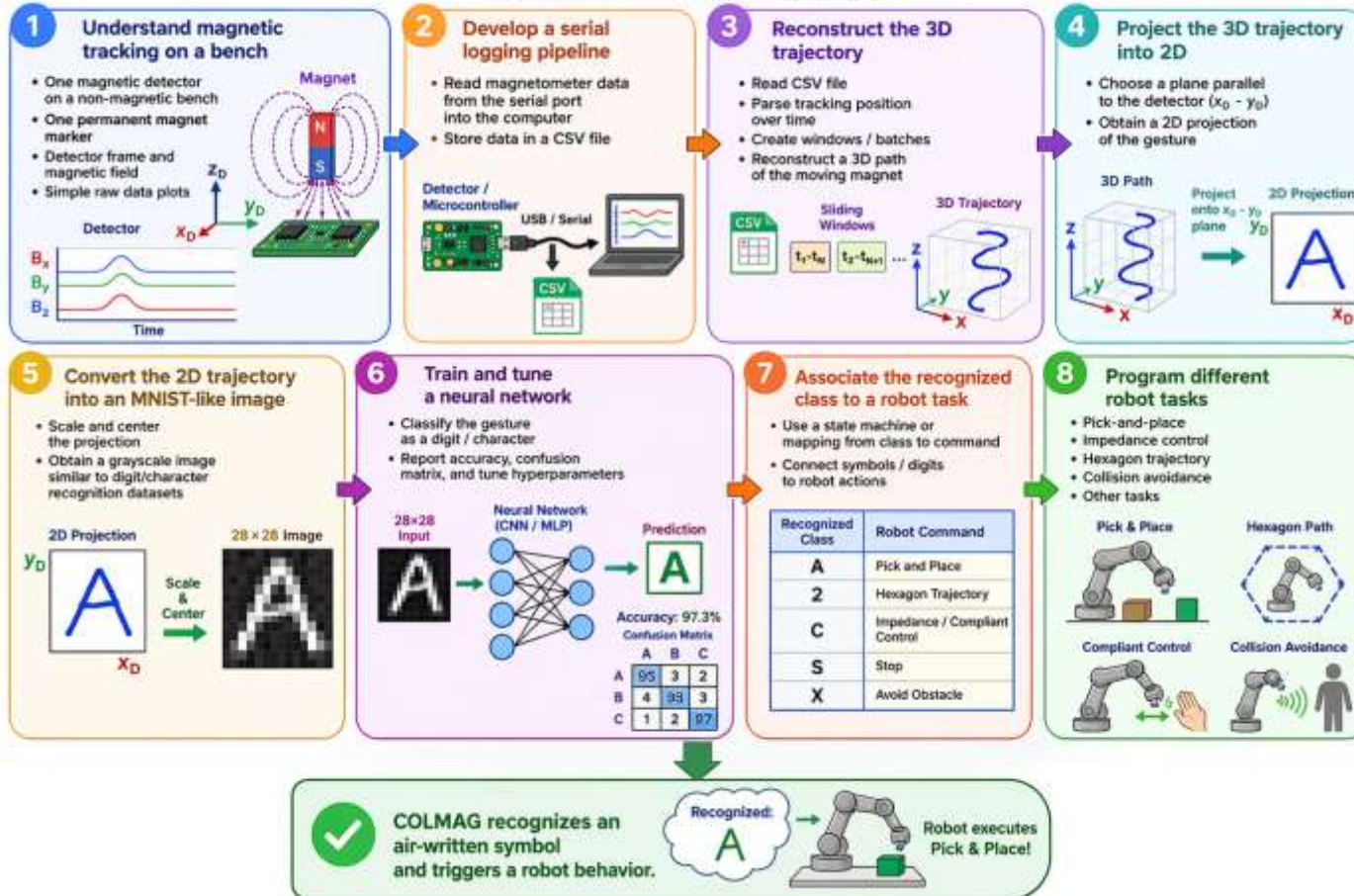
The robot senses magnetic fields with onboard detectors.
You write in the air with a small magnet.

Writing the letter "A" triggers the robot.



COLMAG Work Plan

From magnetic sensing to robot actions



Agenda

Last Week – Recap

- Signal processing on magnetometer data
- 3D trajectory reconstruction
- Projection onto 2D plane
- Neural network for gesture classification
- Graphic User Interface

Today – Introduction to ROS

- What is ROS? Architecture & Philosophy
- ROS 1 vs ROS 2 — quick overview
- Core concepts: nodes, topics, messages...
- Publisher & subscriber in Python
- Your Task

What is ROS

"Robot Operating System (ROS) represents a flexible framework, providing various tools and libraries to write robotic software. It offers several powerful features to help developers in such tasks as message passing, distributing computing, code reusing, and implementation of state-of-the-art algorithms for robotic applications"

Plumbing

Nodes communicate over topics & services. The ROS Master manages all connections.



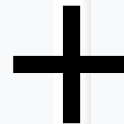
Tools

Extensive toolset to configure, debug, visualize, log, and test your application.



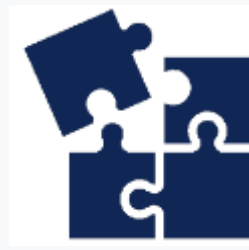
Capabilities

Large library of robot-agnostic packages: drivers, controllers, mapping, planning...



Community

Thousands of open-source packages on GitHub. Active forums, Q&A, tutorials.



ROS Philosophy



Peer-to-peer

Nodes communicate directly after discovery via the Master. No bottleneck — the Master only sets up connections, data flows directly.



Distributed

Nodes can run on different computers across a network. The robot's arm, camera, and AI can run on separate machines.



Multi-language

Write nodes in C++, Python, MATLAB, Java — any language that has a ROS client library. They all talk the same protocol.



Light-weight

Each node does one thing well. Simple, focused processes are easier to debug, test, and replace independently.



Open-source

Completely free. Thousands of contributors. If a package exists for your sensor or algorithm, you can use it immediately.

💡 A few links to bookmark — for tutorials, troubleshooting and community support:

wiki.ros.org — official ROS 1 wiki, installation, tutorials, package docs

wiki.ros.org/rospy/Overview — rospy Python library reference

wiki.ros.org/ROS/Tutorials — step-by-step tutorials from zero

robotics.stackexchange.com — broader robotics Q&A, very active

ROS 1 vs ROS 2

Feature	ROS 1 (Noetic)	ROS 2 (Humble / Jazzy)
Communication	ROS Master (centralized)	DDS middleware (decentralized)
Platforms	Ubuntu / Debian only	Ubuntu, Windows, macOS, RTOS
Real-time support	Limited	Improved real-time control
Multi-robot	Limited support	Native support
Build system	catkin (catkin_make)	colcon + ament
End of Life	May 2025 (Noetic)	Active development
Existing packages	Very large ecosystem	Growing, porting in progress

Why We Use ROS 1

The Franka robot and all our lab code are built on ROS 1 (Noetic). Here's why we stay with it:

Franka runs on ROS 1

The franka_ros package for ROS 1 is mature, stable and production-ready. The ROS 2 port exists but is still evolving — for lab use, ROS 1 is the safe choice.

All existing code is ROS 1

All the code in the lab is written for ROS 1. Your own scripts will need to be wrapped into ROS 1 nodes to communicate with the robot.

Massive community & ecosystem

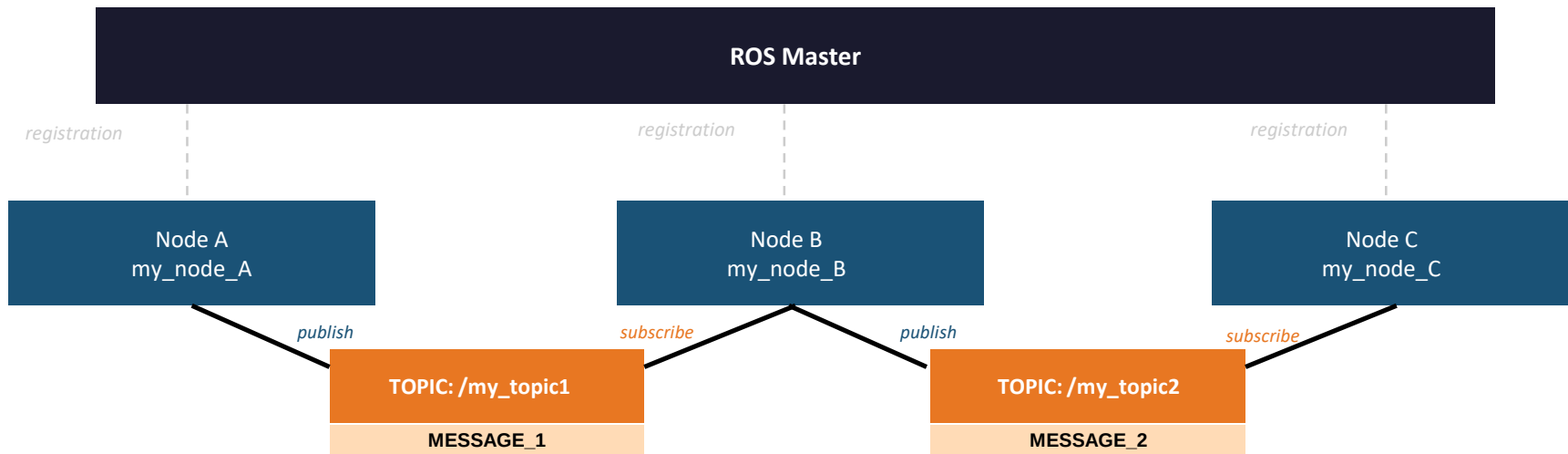
Problems? You'll find the answer immediately on ROS Answers, GitHub, or StackExchange. The ROS 1 community is huge and well-documented.

GitHub repos are mostly ROS 1

The majority of open-source robotics repos (manipulation, planning, perception) target ROS 1.

Plumbing

ROS represents computation as a graph of processes (nodes) connected by edges (topics, services, actions). The Master is the directory that makes it all possible.



💡 *The Master only sets up the connections — actual data flows directly between nodes (peer-to-peer).*

The ROS Master

The Master is like a DNS server for ROS — it knows the address of every node and sets up connections between them. Without it, nodes cannot find each other.

What it does

Name registration

Every node registers its name when it starts up. The Master keeps the directory of all active nodes.

Topic lookup

When a publisher and subscriber share a topic, the Master introduces them — then steps aside. Data never flows through the Master.

Parameter server

The Master also hosts the parameter server — a shared key-value store all nodes can read and write.

1

Node A starts → registers with the Master (name, topics it publishes)

2

Node B starts → registers with the Master (name, topics it subscribes to)

3

Master sees the match → shares Node A's address with Node B

4

Node A and Node B connect directly — Master is no longer involved

 Always start roscore before running any node — if the Master is not running, no node can register.

ROS Nodes

A node is a single executable process. Complex systems are built from many small, specialized nodes communicating with each other.

Key properties of nodes:

- Each node has a unique name (e.g. /camera_node)
- Nodes register with the ROS Master at startup
- One node = one process (separate from all others)
- Multiple nodes can run on different machines over a network

Example — a robot system made of nodes:

- sensor_node — reads data from sensors
- detection_node — runs your gesture classifier
- controller_node — sends commands to robot
- logger_node — saves data to file

```
# Start the ROS Master (always first!)
>> roscore

# Run a node
>> rosrun <package_name> <node_name>

# List running nodes
>> rostopic list

# Info about a node
>> rostopic info /my_node
```

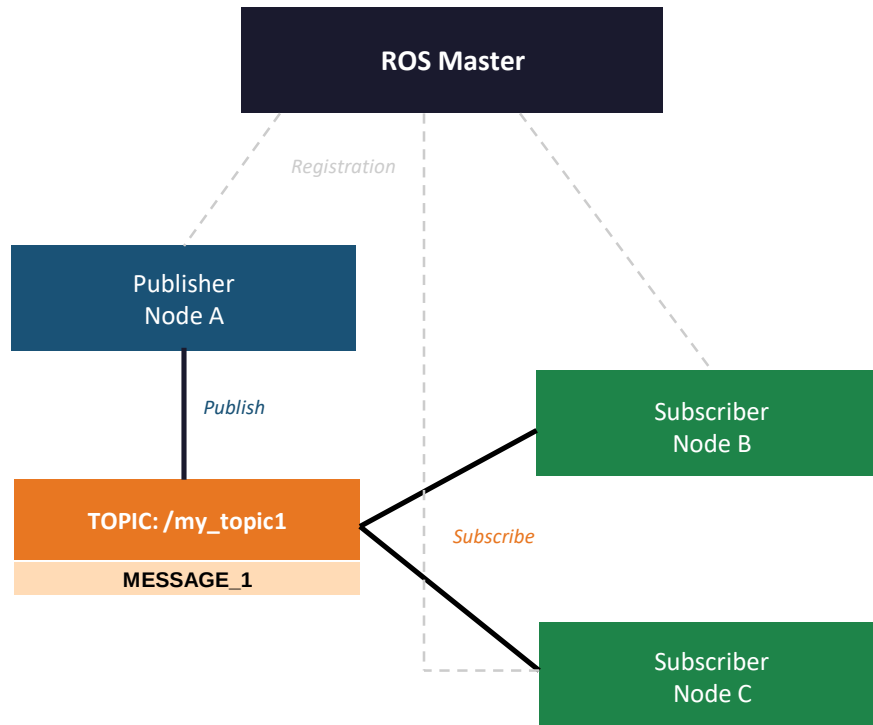
Topics

How nodes communicate:

- A topic is a named channel (e.g. `/joint_states`)
- A publisher sends messages to a topic
- A subscriber receives messages from a topic
- Any number of publishers and subscribers can share a topic
- Communication is asynchronous — publisher doesn't wait for subscribers

Topic CLI commands:

```
rostopic list      # show all topics
rostopic echo /topic # print messages
rostopic info /topic # pub/sub info
rostopic hz /topic  # message frequency
```



ROS Messages — Standard Types

Every topic has a message type. Here are the most common ones you'll use:

Package	Message Type	Content	Typical Use
std_msgs	Bool	bool data	Flags, switches
std_msgs	String	string data	Text, labels, commands
std_msgs	Float64	float64 data	Single numeric value
std_msgs	Int32	int32 data	Integer values
std_msgs	Float64MultiArray	float64[] data	Arrays of numbers (e.g. joint angles)
geometry_msgs	Point	float64 x, y, z	3D position
geometry_msgs	Pose	Point position Quaternion orientation	Full pose (position + orientation)
geometry_msgs	PoseStamped	Header + Pose	Pose with timestamp & reference frame
sensor_msgs	JointState	name, position, velocity, effort	Robot joint states

Workspace & Packages

Catkin Workspace

All your ROS code lives inside a catkin workspace. The workspace has a standard folder structure:

```
catkin_ws/  
├── src/      ← your packages go here  
├── build/    ← auto-generated  
└── devel/    ← compiled targets & setup.bash
```

Create & build the workspace:

```
>> mkdir -p ~/catkin_ws/src  
>> cd ~/catkin_ws  
>> catkin build  
>> source devel/setup.bash
```

⚠️ Run source devel/setup.bash after every build!

Creating a Python Package

A ROS package is the unit of code organization. Create one with:

```
>> cd ~/catkin_ws/src  
>> catkin_create_pkg my_package rospy std_msgs  
geometry_msgs
```

Your package structure:

```
my_package/  
├── CMakeLists.txt ← minimal config  
├── package.xml    ← dependencies  
└── scripts/       ← your Python nodes  
    └── my_node.py
```

Make scripts executable:

```
>> chmod +x scripts/my_node.py
```

CMakeLists.txt — Python Nodes

CMakeLists.txt is a configuration file that tells catkin how to build your package — what to compile, what to install, where to find things. For Python nodes there is no compilation — but you still need to tell catkin where your scripts are so that rosrund can find them.

- 1 Open CMakeLists.txt in your package and find (or add) this block:

```
catkin_install_python(PROGRAMS
  scripts/talker.py
  scripts/listener.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
```

- 2 Add a new script? Just add a line inside the same block — one line per script:

```
catkin_install_python(PROGRAMS
  scripts/talker.py
  scripts/listener.py
  scripts/my_new_node.py ← add your script here
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
```

- 3 Run `catkin make` to register the changes. Then: `source devel/setup.bash`

Python Node – Publisher (talker.py)

 Source: [wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber\(python\)](http://wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber(python))

```
#!/usr/bin/env python3
import rospy
from std_msgs.msg import String

def main():
    rospy.init_node('talker')
    rate = rospy.Rate(10) # 10hz
    pub = rospy.Publisher('/chatter', String, queue_size=10)
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()

if __name__ == '__main__':
    try:
        main()
    except rospy.ROSInterruptException:
        pass
```

 Add to CMakeLists.txt: `catkin_install_python(PROGRAMS scripts/talker.py
DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION})`

1

Shebang — tells the OS to use Python3 to run this file

2

Import rospy — the Python ROS client library

3

Import the String message type from std_msgs

4

init_node() — register with the Master.

5

Publisher(topic, msg_type, queue_size) — declare what we publish

6

Rate(10) — loop will run at 10 Hz

7

is_shutdown() — returns True on Ctrl+C; keeps the loop clean

8

loginfo() — prints to terminal, log file and rosout

9

publish() — send the message

10

ROSInterruptException — caught when Ctrl+C interrupts a sleep()

Python Node – Subscriber (listener.py)

[Source: wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber\(python\)](http://wiki.ros.org/ROS/Tutorials/WritingPublisherSubscriber(python))

```
#!/usr/bin/env python3
import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s",
data.data)

def main():
    rospy.init_node('listener')

    rospy.Subscriber("/chatter", String, callback)

    # spin() keeps python from exiting until node is stopped
    rospy.spin()

if __name__ == '__main__':
    main()
```

- 1 `callback(data)` — called automatically on each incoming message
- 2 `data.data` — access the content of the String message
- 3 `get_caller_id()` — returns this node's full name (e.g. `/listener_1234`)
- 4 `init_node()` — register with the Master
- 5 `Subscriber(topic, msg_type, callback)` — register the callback. Note: pass the function, not its result (no parentheses!)
- 6 `spin()` — blocks here, keeps the node alive and processes callbacks until Ctrl+C

Publisher vs Subscriber — key structural difference: the publisher has an explicit while loop that you control. The subscriber has no loop — `spin()` gives control to ROS, which calls your callback whenever a message arrives.

Run order: Terminal 1 → `roscore` Terminal 2 → `roslaunch pkg talker.py` Terminal 3 → `roslaunch pkg listener.py`

Ros Launch

Instead of opening 5 terminals and running `roslaunch` 5 times, a single `.launch` file starts everything:

`my_package/launch/start.launch`

```
<launch>

  <!-- Start the sensor reader node -->
  <node pkg="my_package"
        type="sensor_node.py"
        name="sensor_node"
        output="screen"/>

  <!-- Start the processing node -->
  <node pkg="my_package"
        type="processing_node.py"
        name="processing_node"
        output="screen"/>

  <!-- Start the publisher -->
  <node pkg="my_package"
        type="publisher_node.py"
        name="publisher_node"
        output="screen"/>

</launch>
```

Key attributes:

`pkg`

package name

`type`

script filename (must be executable)

`name`

node name in the ROS graph

`output`

"screen" prints logs to terminal

Run with:

```
roslaunch my_package start.launch
```

💡 *roslaunch automatically starts roscore — you don't need to run it separately.*

Essential CLI Commands

Core	Nodes	Topics	Messages
<pre>roscore</pre> Start the ROS Master (always first)	<pre>rostopic list</pre> List all running nodes	<pre>rostopic list</pre> List all active topics	<pre>rostopic list</pre> List all message types
<pre>roslaunch <pkg> <node></pre> Run a single node	<pre>rostopic info /node_name</pre> Details about a node	<pre>rostopic echo /topic</pre> Print messages on a topic	<pre>rostopic show std_msgs/Float64</pre> Show message structure
<pre>roslaunch <pkg> <file>.launch</pre> Launch multiple nodes from file	<pre>rostopic kill /node_name</pre> Stop a node	<pre>rostopic info /topic</pre> Show publishers/subscribers	
<pre>roslaunch record <topic></pre> Record specific topic		<pre>rostopic hz /topic</pre> Measure message frequency	
		<pre>rostopic pub /topic type data</pre> Manually publish a message	

Your Task

Environment Setup – Option A : Virtual Machine

❓ What is a Virtual Machine? A virtual machine is a complete computer running inside your computer — it has its own operating system, its own disk space, and its own software. Think of it as a separate Ubuntu laptop that lives inside your Mac. Everything you install and create inside the VM stays there permanently, completely isolated from your Mac.

1 Install VirtualBox or similars (Vmware, Parallels...)

2 Download Ubuntu 20.04 ARM64 Server ISO (Mac users) – AMD64 Desktop (Windows users)
ubuntu.com/download/server/arm — use the ARM64 version (no longer desktop ISO exists for ARM)

3 Create VM & install Ubuntu
Allocate ≥ 4 GB RAM, ≥ 30 GB disk. Boot from ISO and complete the installation

4 Install desktop environment (Mac users only)
`sudo apt install ubuntu-desktop && sudo reboot`

5 Install ROS Noetic
Follow wiki.ros.org/noetic/Installation/Ubuntu — standard installation

Environment Setup – Option B: Docker

 **What is Docker?** A container is a lightweight isolated environment with everything pre-installed — think of it as a mini Ubuntu with ROS already inside, running on your Mac. Your work files live on your Mac and are shared with the container in real time.

1 Install Docker Desktop

```
brew install --cask docker # or download .dmg from: docker.com/products/docker-desktop
```

2 Create a workspace folder on your Mac

```
mkdir -p ~/Desktop/ros_ws
```

3 Run the setup script and follow the instructions

Asks: workspace path, ROS version (choose 1), container name.

```
bash docker_creation_mac.sh
```

4 Inside the container — source ROS (once per session)

```
source /opt/ros/noetic/setup.bash
```

5 Next sessions — reconnect to the existing container

```
bash docker_connection_mac.sh
```

 *Your Mac folder and /home/dev/inside the container are the same — edit on Mac, run with ROS inside the container.*

From Python Script to ROS Node

You already have Python scripts that process data. Here's how to turn them into ROS nodes:

BEFORE — standalone script

```
# your_script.py
import numpy as np

data = read_from_sensor()    # hardcoded input
result = process(data)
print(result)                # hardcoded output
```



AFTER — ROS node

```
#!/usr/bin/env python3
import rospy, numpy as np
from std_msgs.msg import Float64MultiArray

def callback(msg):
    data = np.array(msg.data)    # input from topic
    result = process(data)       # your algorithm
    out = Float64MultiArray()
    out.data = result.tolist()
    pub.publish(out)             # output to topic

rospy.init_node('my_node')
pub = rospy.Publisher('/result',
    Float64MultiArray, queue_size=1)
rospy.Subscriber('/sensor_data',
    Float64MultiArray, callback)
rospy.spin()
```

The 3 changes you need to make:

- ① Replace hardcoded input → subscribe to a topic
- ② Replace print/file output → publish to a topic
- ③ Wrap in `init_node()` + `spin()` — that's it!



Questions?

Aries Lab – Technical University of Munich

F. Masiero, E. Aimi, L. Borriello, Prof. L. Masia